

# Introduction à ROS2

Septembre 2026



Clément Pène - Ingénieur de recherche au LAAS

# SOMMAIR

E

.....

- Pourquoi ROS/ROS2 ?
- L'équation ROS2
- Démo TD1
- Organisation des programmes sous ROS2
- programmer avec ROS2
- ROS2 control



# organisation des cours

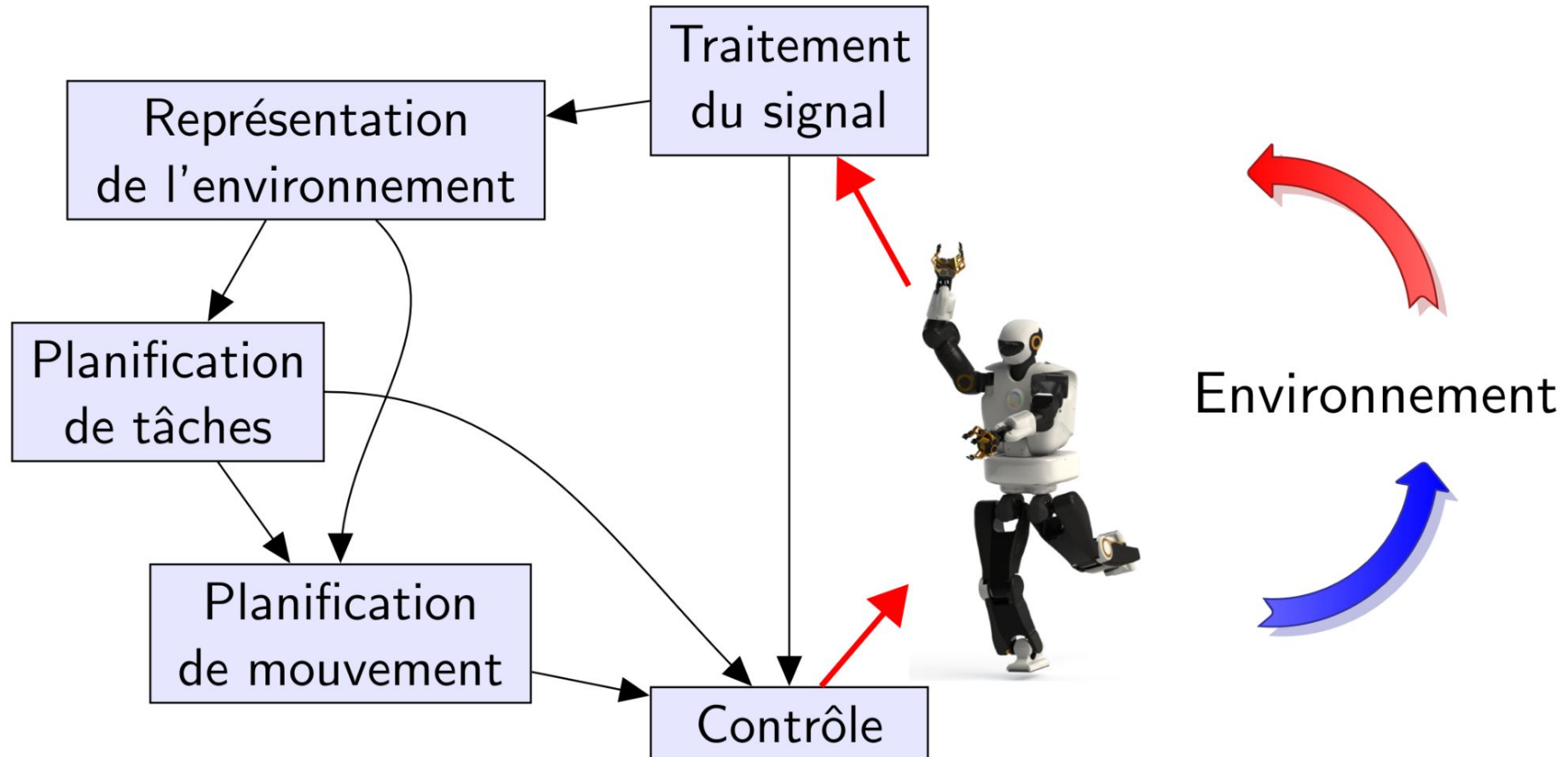
---

- CM (1h15) - introduction à ROS2 - mardi 29 14h - 15h15
- TD1 et 2 - Prise en main des concepts avec turtlesim
- TD3 et 4 - Écrire ses premières nodes
- TD5 - introduction à ros2 control
- TP1 - simulation gazeo, turtlebot, tiago
- TP2 - simulation gazeo, turtlebot, tiago
- TP3 - navigation sur tiago (vrai robot)

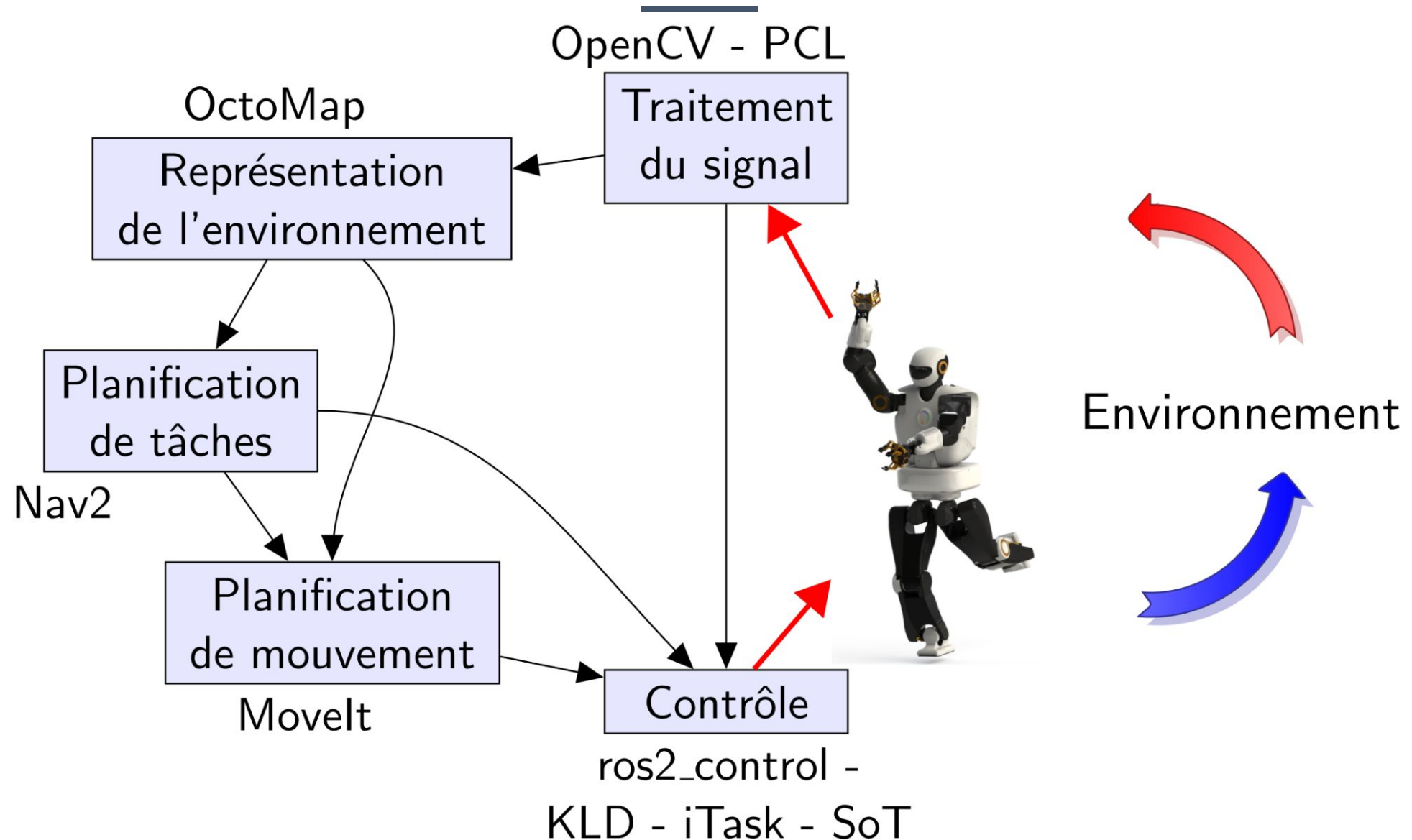


# Pourquoi ROS/ROS2

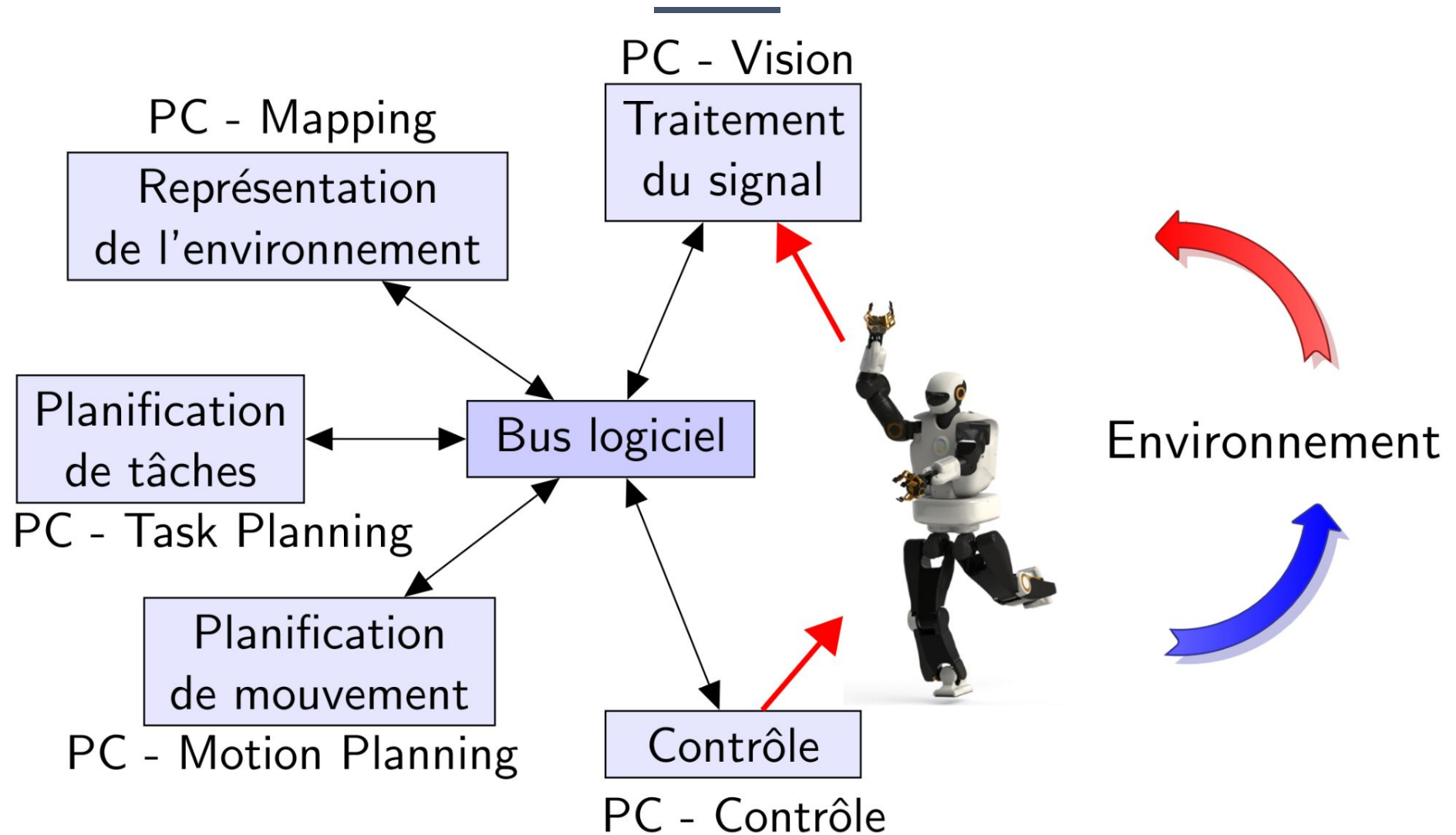
# Motivations - application robotique



# Motivations - application robotique - outils



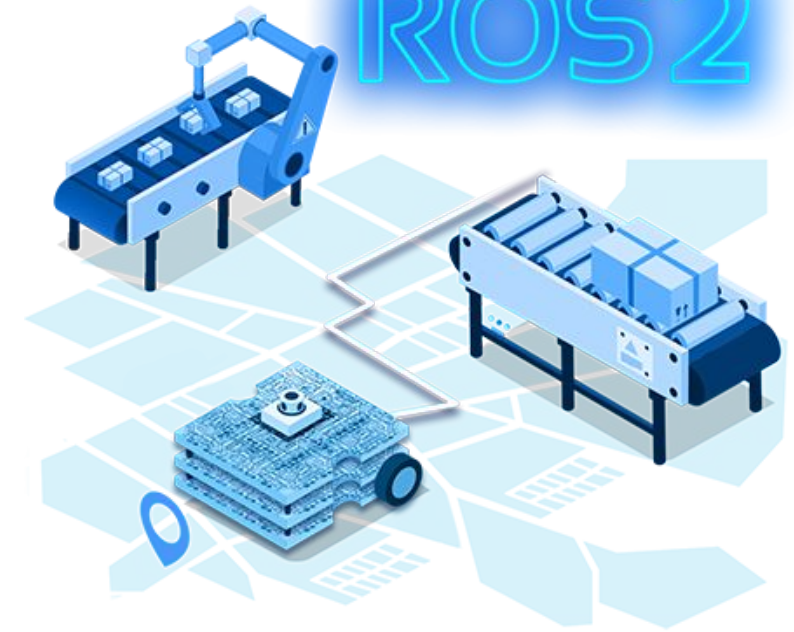
# Motivations - application robotique distribuée



# Solution : ROS

(Robot Operating System)

ROS2



## Définition

Environnement open source de développement logiciel  
*collaboratif*

Middleware

## Objectif et utilisation

Créer un standard en robotique permettant la  
réutilisation des briques de bases composant un système  
robotique

Recherche (partager efficacement les avancées  
scientifiques)

Education (architecture robotique standard)

Entreprise (rapidité de développement)

## Inconvénients

Temps réel non garanti  
Surcoût de communication

Courbe d'apprentissage

Gestion des versions

# Exemples d'utilisations possibles

Support des systèmes multi-robots impliquant des réseaux non-fiables



Plateformes hardware / multi-OS



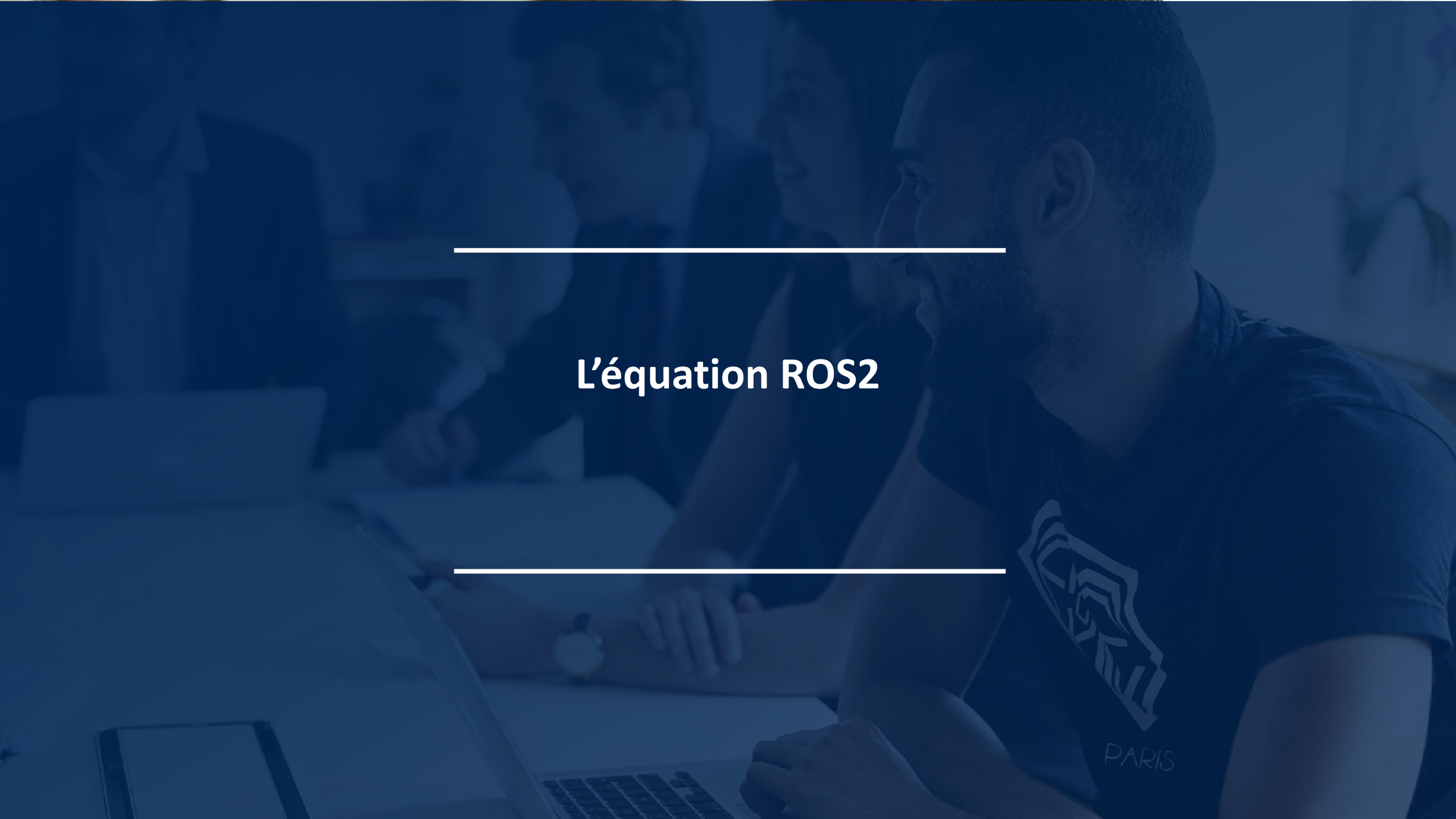
micro-contrôleurs

Parties de robots



Robots complets





# L'équation ROS2

# L'équation ROS2



# La plomberie (DDS)

(Data Distribution Service)

---

## Rôle

- Publication / souscription à la transmission de messages anonymes (Standard industriel de communication)
- Enregistrer et rejouer des messages
- QoS (Quality of Service) : Réglage de la transmission (fiabilité, durée de vie, historique)
- RMW (ROS Middleware interface) : couche qui gère l'implémentation et l'utilisation de la plomberie

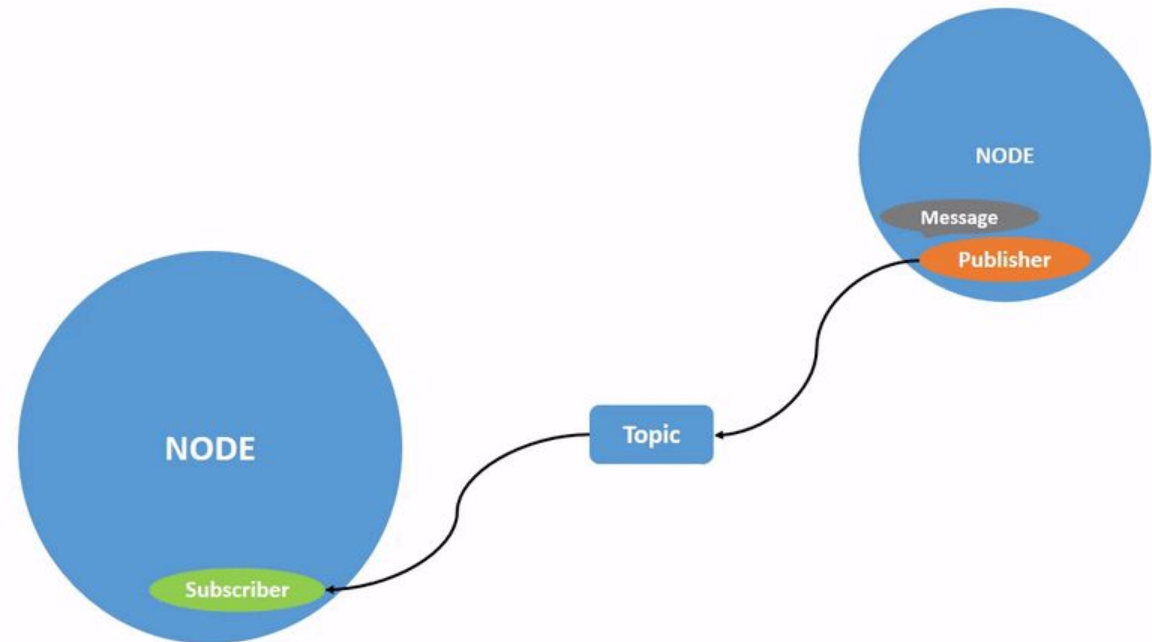
## Autres alternatives

- Fast DDS (eProsima) -> choix par défaut sous Jazzy
- Cyclone DDS (Eclipse)
- Zenoh (pas du DDS)

# Topics et nodes

---

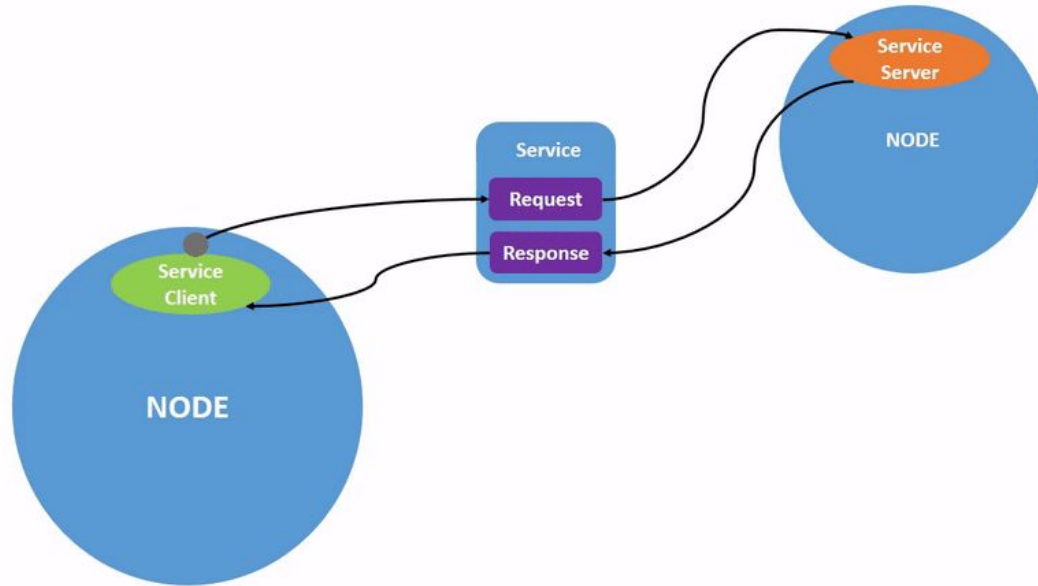
- | Node : programme
- | Topic : canal de transmission de messages typés
- | Paramètre : réglage qui appartient à un node et qui peut être lu ou modifié pendant qu'il tourne



# Services et Actions

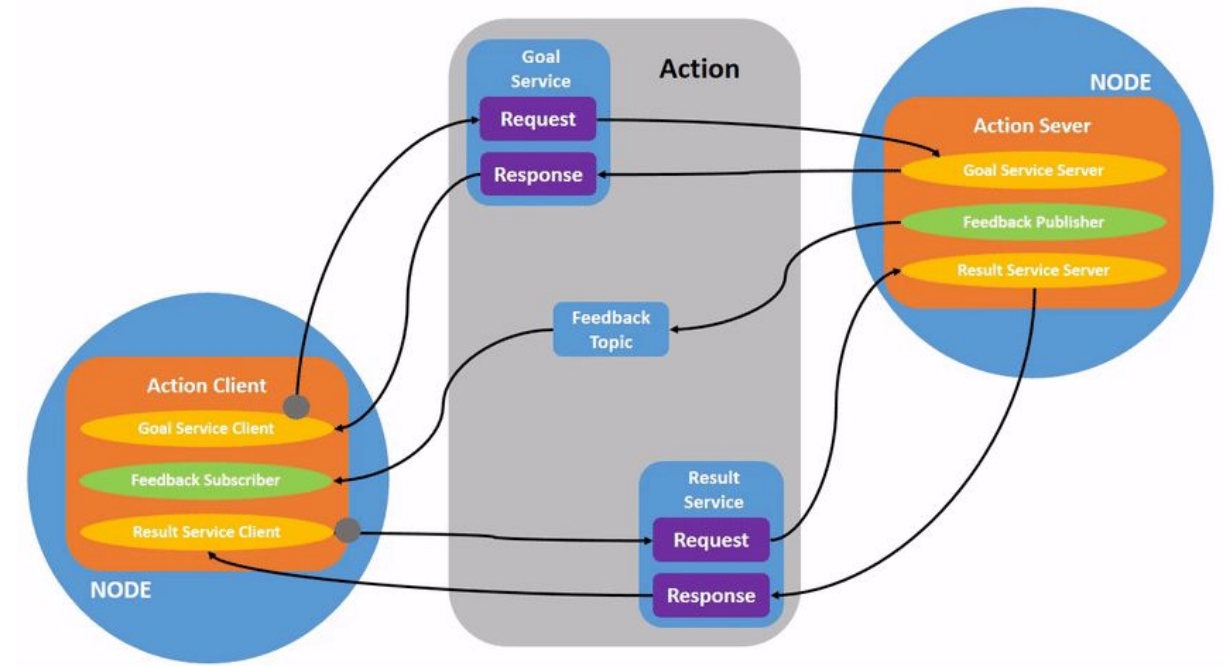
## Services

Requête à un node. Ponctuel.



## Actions

But long à atteindre



# Les outils

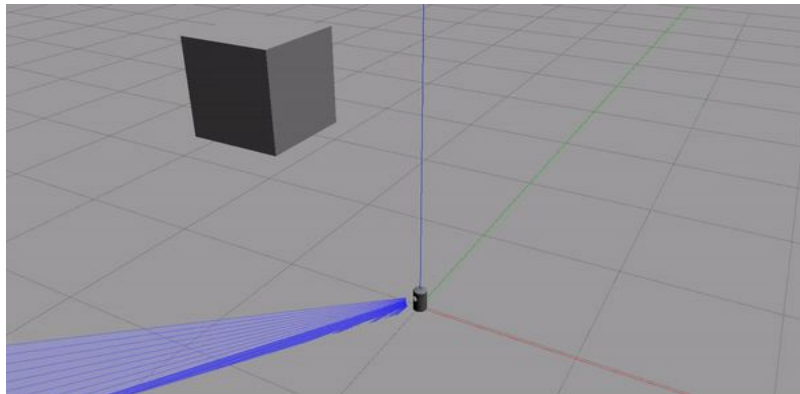
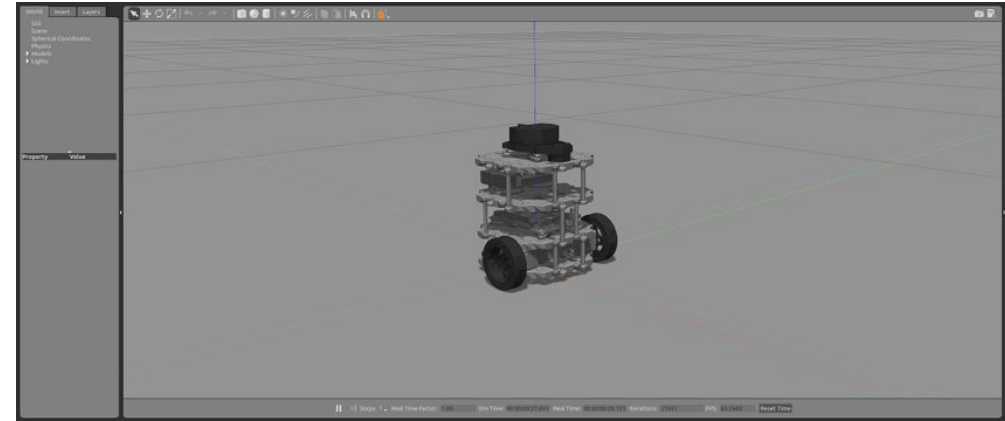
---

- **ros2** : inspecter et piloter un système en ligne de commande (TD1)
- **rqt\_graph** : Visualisation du système (TD1)
- **rqt\_plot / rqt\_console** : Affichage des courbes en direct, journal des messages
- **ros2 bag** : Enregistrer et rejouer les données (bonus)
- **colcon** : outil de compilation (TD2)
- **rviz2** : Visualisation 3D (TD5)

# Outils de simulation

## Gazebo

Simulateur 3D, cinématique, dynamique et multi-robot (TP)



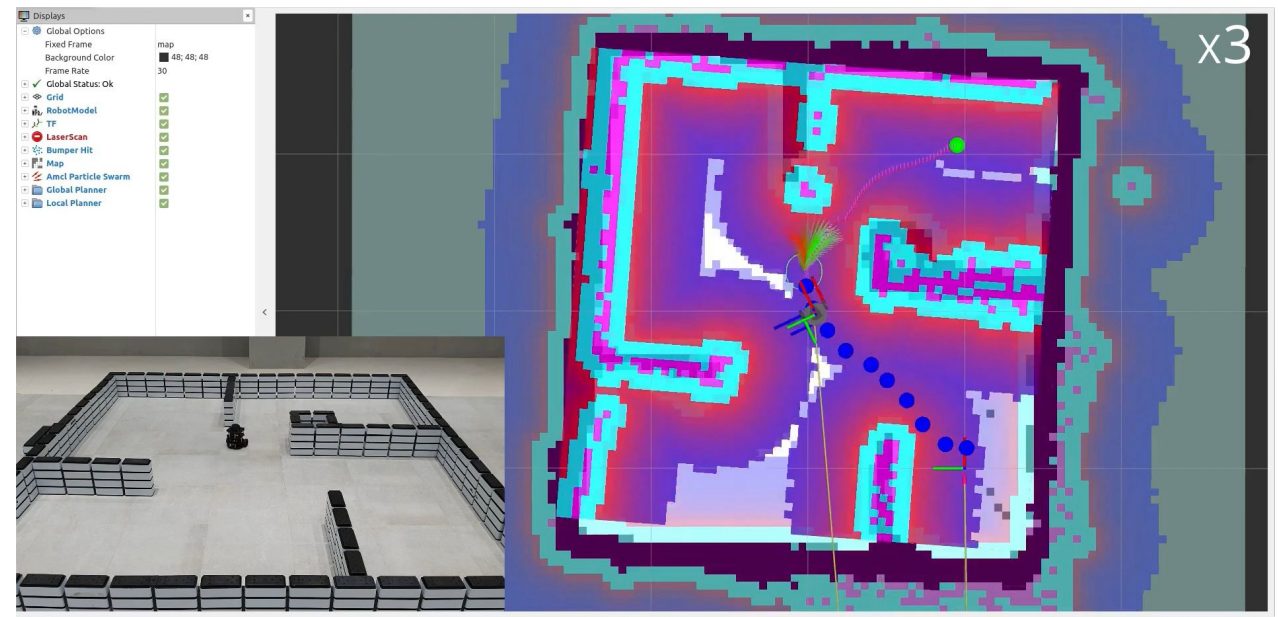
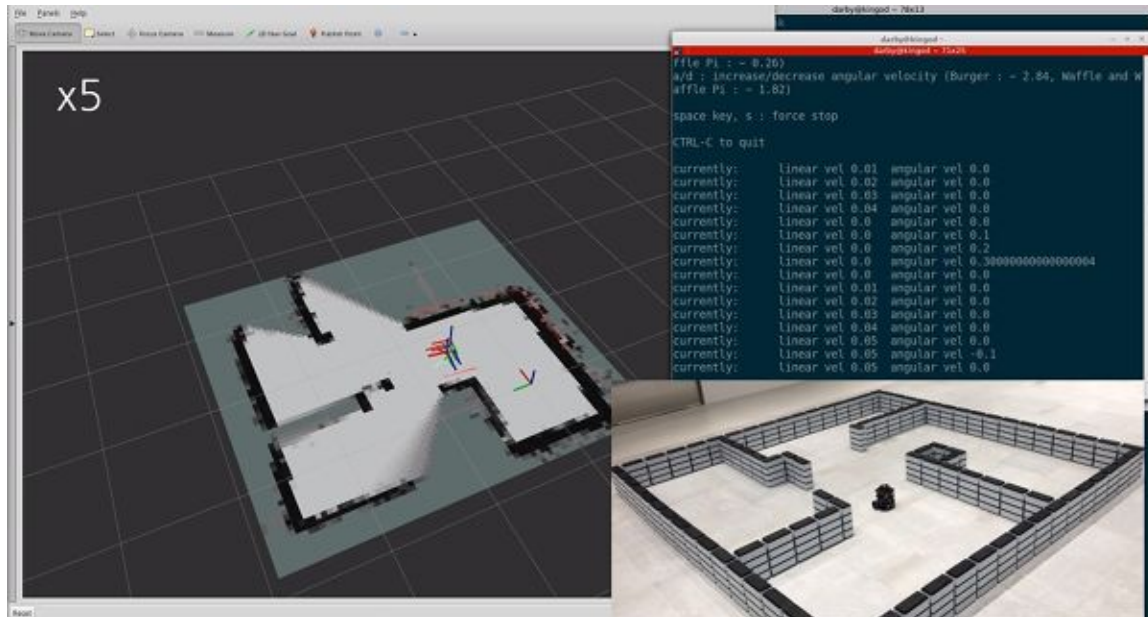
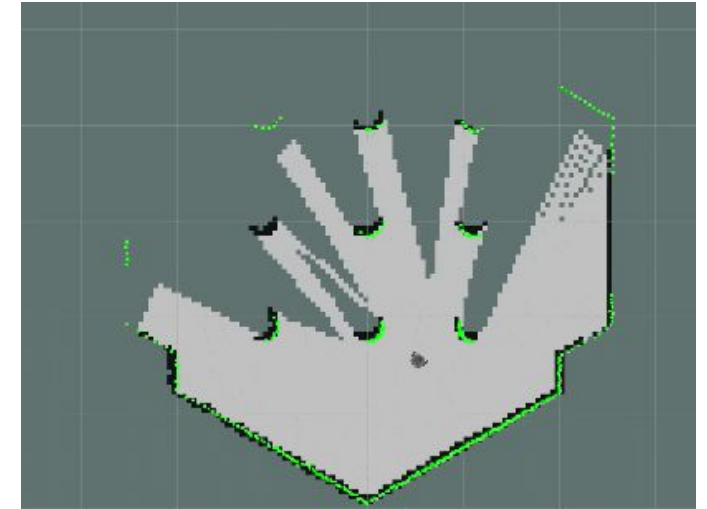
simulation d'un lidar avec un bruit gaussien



# Capacités

(Ce que l'on peut faire avec des algorithmes déjà écrits)

- Manipulation (Moveit)
- Localisation
- Cartographie / SLAM (Simultaneous Localization and Mapping)
- Navigation (nav2)



# Ecosystème

---

- Définitions de messages standards (type du message)
- Librairie pour la géométrie des robots (tf2)
- Langage de description des robots (urdf)
- Diagnostics
- Nombreux robots disponibles
- Infinité de paquets disponibles (pas tous de bonne qualité)



---

## Démo TD1

---

# Ressources utilisées

---

- [documentation officielle](#)
- polycopié d'Olivier p. 95 à 118
- VM disponible ici : <http://homepages.laas.fr/gsaurel/vm.ova>
- pour les curieux, voici le github de la VM : <https://github.com/nim65s/mfja-config>

# Configuration de l'environnement

---

- aller dans le dossier `INSA_ros2_env`
- si c'est la première fois : lancer `direnv allow`
- vous pouvez regarder le fichier `.envrc` qui est chargé par `direnv` à chaque fois que vous entrez dans ce dossier - il contient notamment des variables d'environnement propres à ROS2

# Exercices

---

- Prendre en main l'outil en ligne de commande `ros2` pour utiliser des nodes, topics, services et actions.
- En modifiant les variables d'environnement de ROS, prenez le contrôle des tortues de vos voisins
- Effectuez un cercle avec une de vos tortues en une seule commande
- Services : utiliser `spawn` / `kill` et `set_pen`
- Paramètres
- Actions
- Bonus : utiliser `ros2bag` et les fichiers `launch`



---

# Organisation des programmes sous ROS2

---

# Architecture software - exemple

Environnement ROS



Pkgs



→ Paquet permettant de gérer le déplacement écrit **en C**

→ Paquet qui gère les données de distance UWB écrit **en RUST**

→ Paquet qui gère les données de distance des capteurs magnétiques écrit **en Python**

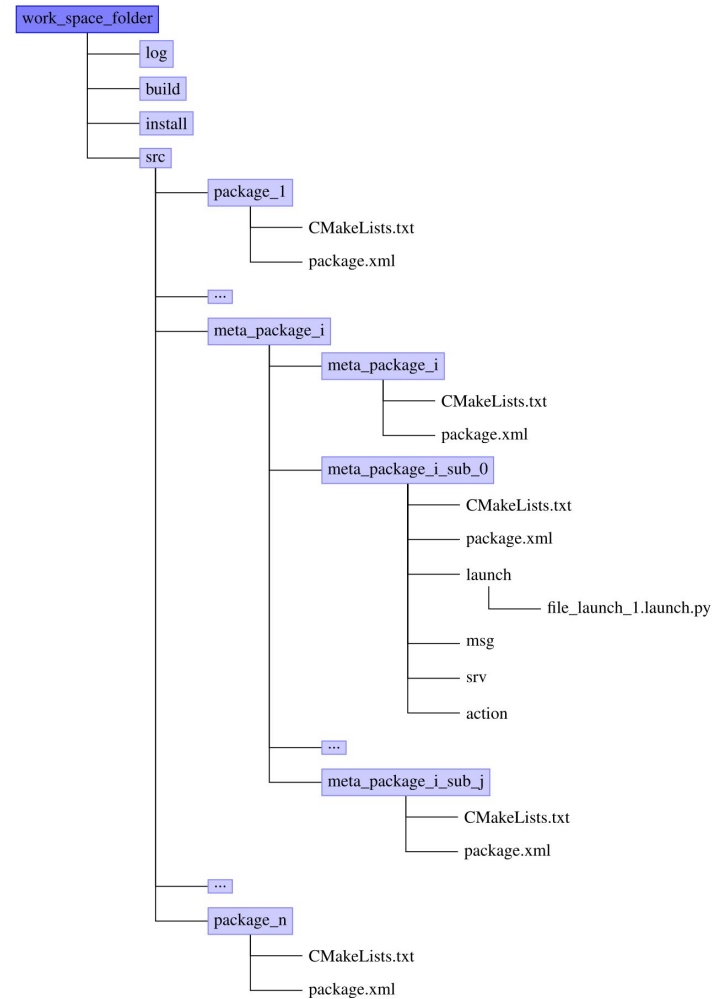
---

**Découpage en paquets**

**Un paquet par fonctionnalité (ex : déplacement ou mesure)**

**Réutilisable**

# Architecture software complete





---

**TD2**

---

# Ressources utilisées

---

- [documentation officielle](#)
- [API de rclpy](#)
- polycopié d'Olivier p. 123 à 140
- build avec colcon –symlink-install (ne pas build dans le dossier src/)
- ne pas oublier de sourcer les nouveaux paquets compilés : source install/setup.bash

# Exercices

---

- Refaire l'exercice du TD1 avec une node (cercle avec une tortue) - ajouter le rayon du cercle en paramètre.
- Exercice service : écrire un service qui compte le nombre de tortues vivantes (CountTurtles)
- Exercice service : écrire un service qui mesure la distance et l'angle à parcourir entre un point donné et une tortue (DistanceToGoal)
- Exercice action : écrire une action permettant d'atteindre une coordonnée donnée



# ROS2\_control